

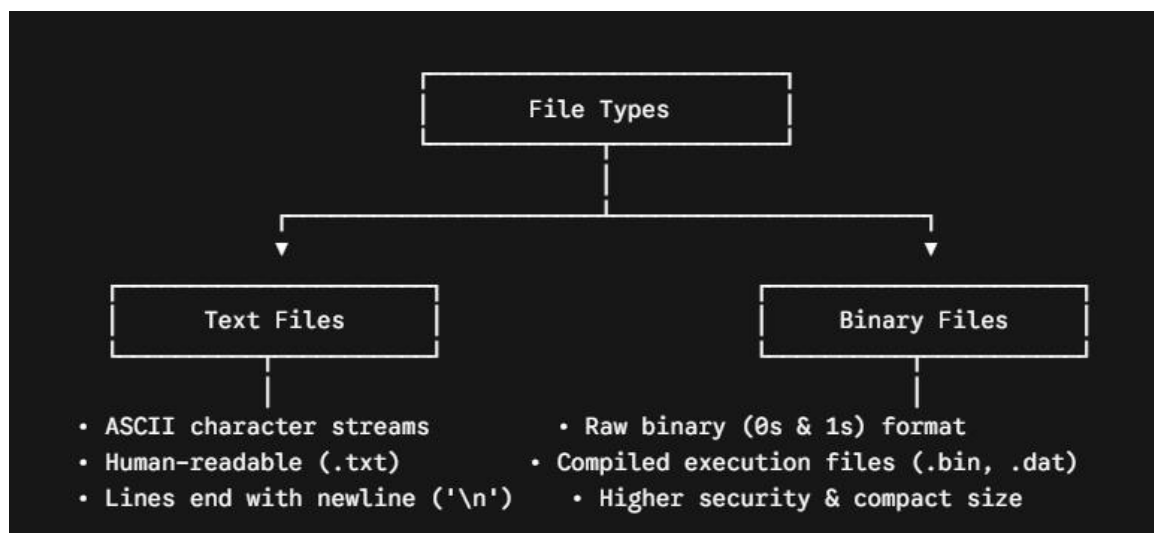
1. Introduction to File Handling in C

Why File Handling?

In standard C programs, keyboard inputs and terminal outputs rely on **volatile primary memory (RAM)**. Once the program terminates, all runtime data is erased.

Files store data permanently on secondary storage devices (HDD/SSD) as a sequence of bytes, enabling data persistence across program executions.

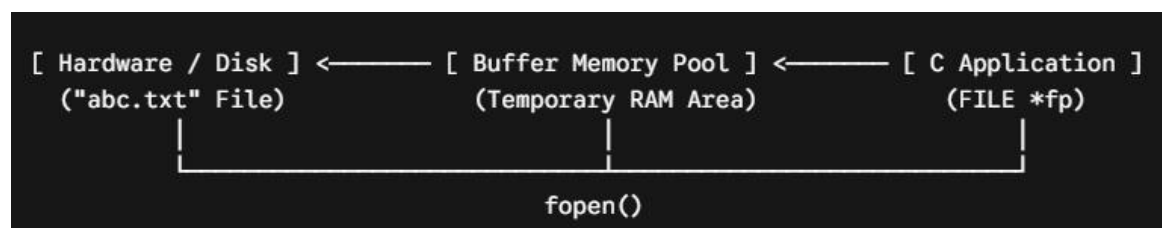
Classification of Files



2. The FILE Pointer and Memory Buffering

Directly reading from or writing to persistent disk storage for every single character is extremely slow. C optimizes file operations by using **Memory Buffering** managed through a specialized pointer called a FILE pointer (FILE *fp).

Architecture of File Buffering



The FILE Pointer: Formally declared as FILE *fp;. It holds the address of a system structure managing the active file stream.

The Buffer Area: A temporary RAM space allocated automatically when a file opens. Data sent to a file collects in this buffer first.

Flushing: When the buffer fills up or when `fclose(fp)` is invoked, its contents are written to the disk—a process called **flushing**. The buffer can also be manually cleared at any time using `fflush(fp)`.

3. Opening & Closing Files: File Modes

Files are initialized using `fopen()` and released using `fclose()`:

`FILE *fp = fopen("filename.txt", "mode");` ----> **Returns NULL on failure**
`fclose(fp);` -----> **Flushes buffer and closes stream**

Summary Matrix of Standard File Modes

Mode String	Primary Purpose	File Exists Behavior	File Doesn't Exist Behavior	Starting Cursor Location
"r"	Read-only	Opens successfully	Returns NULL error	Beginning of File
"w"	Write-only	Erases existing content	Creates a new file	Beginning of File
"a"	Append-only	Preserves content; appends to end	Creates a new file	End of File
"r+"	Read + Write	Preserves content; allows overwrite	Returns NULL error	Beginning of File
"w+"	Write + Read	Erases existing content	Creates a new file	Beginning of File
"a+"	Append + Read	Preserves content; appends new data	Creates a new file	End of File

Fundamental Input/Output File Functions

Writing to Files

fputc(ch, fp): Writes a single character ch to the file.

fputs(str, fp): Writes a null-terminated string str to the file.

fprintf(fp, "format", ...): Writes formatted, mixed-type data (e.g., integers, strings, floats) to the file stream.

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    FILE *fp = fopen("abc.txt", "w");
    if (fp == NULL) -----> Always guard against NULL pointers
    {
        printf("Error: Unable to open file.\n");
        exit(1);
    }

    int age = 20;
    char name[] = "Gaurav";

    fputc('A', fp); -----> Single character
    fputs("\nUser Details:\n", fp); -----> Plain string
    fprintf(fp, "Name: %s, Age: %d", name, age); -----> Formatted output

    fclose(fp); -----> Closes file and flushes buffer
    return 0;
}
```

Reading from Files

fgetc(fp): Reads a single character from the current cursor position. Returns EOF when it reaches the end of the file.

fgets(buffer, n, fp): Reads a line or string up to \$n - 1\$ characters, reserving the last byte for the null terminator (\0). It stops automatically upon encountering a newline (\n).

fscanf(fp, "format", ...): Reads formatted data into variables.

```

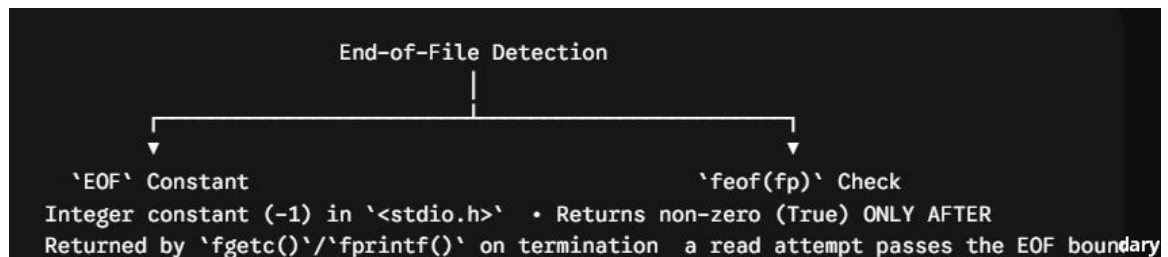
#include <stdio.h>
#include <stdlib.h>
int main()
{
    FILE *fp = fopen("abc.txt", "r");
    if (fp == NULL)
    {
        exit(1);
    }
    char buffer[100];
    while (fgets(buffer, sizeof(buffer), fp) != NULL) ----> Method: Reading line-by-line
    {                                           using fgets()
        printf("%s", buffer);
    }

    fclose(fp);
    return 0;
}

```

End of File (EOF), feof(), and ferror()

When reading files, operations must detect when the data stream ends.



⚠ **feof() vs ferror():** Functions like `fgetc()` return EOF both at the end of a file **and** when an I/O error occurs.

Use `feof(fp)` to explicitly check if the stream reached the end of the file.

Use `ferror(fp)` to explicitly check if an I/O operation failed due to a system error.

6. Random Access & Positioning Functions

rewind()

Resets the file position indicator back to the **very beginning** of the file stream.

rewind(fp); -----> **Equivalent to fseek(fp, 0, SEEK_SET)**

ftell()

Returns the current offset byte position of the file pointer relative to the beginning of the file. Returns a long int.

```
FILE *fp = fopen("abc.txt", "r");
```

```
fseek(fp, 0, SEEK_END); -----> Move cursor to end of file
```

```
long int length = ftell(fp); -----> Calculates total file size in bytes!
```

```
printf("File size: %ld bytes\n", length);
```

fseek()

Moves the file pointer cursor to a specific location by applying a byte offset relative to a reference position.

```
int fseek(FILE *fp, long offset, int origin);
```

The Three fseek() Origin Flags (origin)

Flag Constant	Numeric Equivalent	Reference Position
SEEK_SET	0	Beginning of the file
SEEK_CUR	1	Current position of the file pointer
SEEK_END	2	End of the file

Cursor Movement Visualizer

File Content: "I am Gaurav Kandpal"

Index Bytes: 0123456789...

fseek(fp, 6, SEEK_SET); ——> Moves cursor 6 bytes forward from Start ——>
Points to 'G'

fseek(fp, -3, SEEK_CUR); ——> Moves cursor 3 bytes backward from Current —>
Points to 'm'

fseek(fp, -5, SEEK_END); ——> Moves cursor 5 bytes backward from End ——>
Points to 'd'